

AI Service Orchestrator for Informal Economy

Requirements & Scope of Work: Source of Truth

Hackathon: Google Antigravity Hackathon — Challenge 2

Working Days: 5 (May 15 → May 20, 2026)

How to use this document: This is the single source of truth. If something isn't in here, it's out of scope. Before proposing any feature addition, check the "Out of Scope" section first. Disputes about what to build get resolved by re-reading this doc.

1. Executive Summary

1.1 Problem

Pakistan's informal service economy — plumbers, electricians, AC technicians, tutors — operates through fragmented WhatsApp messages, phone calls, and word-of-mouth referrals. This results in:

- Poor service matching (closest provider isn't always best)
- Unpredictable pricing (no transparency)
- No reliable follow-up or quality assurance
- Weak dispute resolution
- Lost opportunity for both customers and providers

1.2 Solution

An agentic AI mobile application that automates the end-to-end service request lifecycle: natural language intent → multi-factor provider matching → dynamic pricing → booking → service execution → feedback loop → dispute handling. Both User and Provider sides exist in a single React Native app with role-switching at login. Google Antigravity orchestrates the agent workflow and provides traceable reasoning at every decision point.

1.3 Why We Will Win This Challenge

Three differentiators that other teams will not match:

1. **Roman Urdu / Urdu first-class support** — most teams will pay lip service; we will make this the demo opener
2. **Visible agentic reasoning** — every decision in the app exposes a "How I decided" panel showing the agent's chain of thought from Antigravity traces
3. **Auto-rescheduling on provider cancellation** — a real agentic moment where the system observes a failure, reasons about alternatives, and acts autonomously without user intervention

1.4 Success Criteria

The submission succeeds if all of the following are true on May 20:

- Mobile app runs and completes the full demo flow end-to-end without crashes
- Demo video (3–5 min) clearly shows: input → intent → ranking → pricing → booking → cancellation → reschedule → completion → feedback
- Antigravity artifacts (workplan, tasks, reasoning logs) are exported and submitted
- README is comprehensive and includes baseline comparison, cost analysis, scalability note
- At least one stress-test scenario is demonstrated end-to-end

2. Locked Decisions

Decision	Value	Rationale
Challenge	C2 — Service Orchestrator	Strong domain fit, achievable scope
Mobile platform	React Native	Cross-platform, team strength
Backend	Node.js + TypeScript	Same language as RN, schema validation built-in
LLM	Google Gemini	Google ecosystem alignment, multilingual strength
Database	PostgreSQL	Relational database with transactions and joins
Maps	Google Maps Platform (Places + Distance Matrix)	Required for distance-based ranking
Hosting (backend)	Google Cloud Run or AWS	Cloud Run if time, ngrok as local fallback if deployment slips
Roles	User + Provider, role-switch at login	Single codebase, dual perspective
Service category	AC technicians only	Depth over breadth (rubric: "prioritize reliable end-to-end over broad but shallow")
Payment	Mock only	Not in rubric
Voice input	Stretch goal	High failure risk on Roman Urdu
Languages supported	English, Urdu, Roman Urdu, code-switched	Rubric requirement
Provider dataset size	30 mock providers	Enough for realistic ranking
Hero stress test	Provider cancellation + auto-reschedule	Agentic intelligence on display
Secondary stress test	Ambiguous input + confidence-based clarify	Hits multilingual robustness

4. Feature Inventory

Features are tagged by priority. The team must complete all **M** (Must-Have) features before touching **S** (Should-Have). **C** (Could-Have) features are stretch goals only after Day 4 is clean. **W** (Won't-Have) features are explicitly excluded and should not be built even if a teammate proposes them.

4.1 User-Side Features

ID	Feature	Priority	Notes
U-1	User email login with role selection	M	Auth
U-2	Chat-based request input (text)	M	react-native-gifted-chat
U-3	Intent extraction with confidence score	M	Gemini structured output
U-4	Intent confirmation card (editable)	M	Shows parsed service, location, time, urgency, budget
U-5	Low-confidence clarification flow	M	If confidence < 0.7, ask a clarifying question
U-6	Provider ranking display (top 3)	M	Cards with name, distance, rating, ETA, price
U-7	"How I decided" reasoning panel per provider	M	Tap to expand, shows agent reasoning
U-8	Dynamic price quote with breakdown	M	Visit fee + distance + urgency + complexity + discount
U-9	Booking confirmation screen	M	Calendar slot, provider details, total price
U-10	Simulated SMS/WhatsApp confirmation card	M	Renders as a system card in chat
U-11	Active booking view (en-route, in-progress)	M	Status timeline
U-12	In-app notification bell + polling	M	Frontend polls every 15s
U-13	Auto-rescheduling on provider cancellation	M	Hero demo moment
U-14	Service completion + feedback form	M	Star rating + text comment
U-15	Dispute submission form	M	Reason dropdown, free text, optional photo
U-16	Booking history	S	List of past + current bookings

ID	Feature	Priority	Notes
U-17	Multi-language toggle (UI text)	S	Just for the demo polish
U-18	Voice input for service request	C	Stretch, if we have remaining time
U-19	Photo upload for problem description	C	Stretch, if we have remaining time

4.2 Provider-Side Features

ID	Feature	Priority	Notes
P-1	Provider role login	M	Auth
P-2	Inbox: incoming requests with accept/decline	M	FE polls the backend API
P-3	Daily schedule view	M	Time-blocked calendar for today
P-4	Active job: customer info, location, checklist	M	Includes "I'm en route" button
P-5	Service completion: photo placeholder, mark done	M	Updates booking status
P-6	Provider Insights screen (earnings + workload)	M	Single screen: today's bookings count, week earnings total, busy-hours chart aggregated from bookings table. Hits provider-side optimization rubric requirement. No AI needed — pure aggregation.
P-7	Cancel booking flow (with reason)	M	Triggers the reschedule hero demo
P-8	Ratings & reviews view	S	Reputation tab
P-9	Dispute response interface	S	Provider's side of dispute resolution
P-10	AI-recommended open slots	S	Demand Forecast Agent feeds this — stretch if time
P-11	Availability toggle (online/offline)	C	Stretch

4.3 Backend / Agentic Features

ID	Feature	Priority	Notes
B-1	Intent Agent	M	NL → structured intent JSON with confidence
B-2	Discovery Agent	M	Query providers by category + location
B-3	Ranking Agent	M	Six-factor scoring with reasoning string
B-4	Pricing Agent	M	Dynamic price with breakdown JSON
B-5	Booking Agent	M	Slot reservation, conflict prevention
B-6	Follow-up Agent	M	Reminder scheduling, en-route updates
B-7	Feedback Agent	M	Rating ingestion, reputation update
B-8	Dispute Agent	M	Reasoning on dispute, recommended resolution
B-9	Reschedule Agent	M	Triggered on cancellation; finds next-best provider
B-10	Demand Forecast Agent	S	For Provider Insights tab
B-11	Agent trace logger	M	Writes every decision to the database for app display
B-12	Baseline ranker (heuristic)	M	For baseline comparison in README
B-13	Scheduling engine with travel buffers	M	Prevents double-booking, adds buffer time

4.4 Cross-Cutting / Infrastructure

ID	Feature	Priority	Notes
I-1	Database models (PostgreSQL schema)	M	Tables for users, providers, availability, bookings, messages, notifications, agent_traces, scheduled_tasks
I-2	Mock provider dataset (30 providers)	M	Curated variety in ratings, specializations, cancellation rates
I-3	API key management (Gemini, Maps)	M	.env, never commit
I-4	Agent trace export script	M	For submission deliverable
I-5	README with all required sections	M	Architecture, schemas, APIs, Antigravity role, setup, assumptions, privacy, cost/latency, scalability, baseline comparison, limitations
I-6	Demo video script + recording	M	3–5 min main demo + 2–3 min Antigravity workflow video

8. Simulation Specifications

The rubric demands action *simulation*, not real execution. Here's exactly what each simulated action looks like.

8.1 Simulated SMS / WhatsApp Confirmation

What happens: The Booking Agent generates a confirmation message string in the user's language and inserts it into the `messages` table with status `simulated_sent` and a foreign key to the booking. The mobile app fetches messages via the polling API and renders them as system cards in the chat UI.

Visual: Looks like a real WhatsApp message bubble inside the app. Has WhatsApp green styling and a small label "Simulated WhatsApp message" so it's clearly a simulation but visually convincing.

8.2 Simulated Calendar Update

What happens: When a booking is confirmed, the relevant row in the `availability` table has its `booking_id` foreign key updated from null to the booking ID, inside the same database transaction that creates the booking. The provider's schedule view fetches availability on next poll and shows the slot as booked.

8.3 Simulated Reminder (1 hour before)

What happens: A row in the `scheduled_tasks` table is inserted with `execute_at = scheduled_start - 1hr`. A backend cron job (or for the demo, a "Skip Forward Time" button) checks this table every 30 seconds and "fires" any task whose time has passed by creating the reminder message in the `messages` table and a corresponding row in the `notifications` table.

Demo trick: For the video, include a "Fast-Forward" debug button that advances simulated time so reminders fire visibly within the 3-min demo.

8.4 Simulated En-Route Update

What happens: When the provider taps "I'm en route" on their app, a row is inserted into the `messages` table with type=en_route and body containing simulated ETA. A notification is also created. The customer app shows this as a system card with a fake map preview (static image of route from provider location to customer, using Maps Static API).

8.5 Simulated Service Completion

What happens: Provider taps "Mark Complete" on the active job screen. Booking status updates to `completed`. A feedback request message is generated and inserted into the `messages` table, and a notification is created. Customer app shows a rating prompt on next poll.

8.6 Simulated Auto-Rescheduling (Hero Demo)

What happens (in order):

1. Provider opens active job and taps "Cancel" with reason "emergency"
2. Booking status updates to `cancelled_by_provider` via API call
3. Backend's cancellation endpoint triggers the Reschedule Agent automatically (in-process, same request lifecycle)
4. Reschedule Agent re-runs ranking on the original intent, excluding the cancelled provider
5. New top-ranked provider is offered the slot
6. If they accept (auto-accept in demo for speed), a new booking is created with status=confirmed
7. Customer receives a notification: "Your provider had an emergency. I've rescheduled with Bilal AC Repair (4.6★, 2.3km, same price, available at the same time)"
8. Customer app shows the new booking card on next poll

Demo time: End-to-end in 8-10 seconds. Looks magical.

8.7 Simulated Dispute Resolution

What happens:

1. Customer submits dispute (reason: "Service incomplete")
2. Dispute Agent reads the booking history, completion checklist, photo placeholder, and reasons about the case
3. Dispute Agent generates a resolution recommendation: refund amount, compensation, reputation impact
4. Resolution is displayed to both parties with clear reasoning
5. For demo: auto-applies after 2 seconds

9. Agentic Workflow Specification

9.1 Agent Topology

The system has 9 agents. They are LLM-powered functions in the Node.js backend, called by an orchestrator function. Each agent has a strict input schema, output schema, and reasoning string.

User Input

↓

	Orchestrator (TypeScript function)	
	- Routes between agents	
	- Manages shared trace state	
	- Handles failures and retries	

↓

1. Intent Agent → structured request, confidence

- ↓ (if confidence < 0.7, return clarify question)
- 2. **Discovery** Agent → candidate providers (raw list)
 - ↓
- 3. **Ranking** Agent → top 3 with **scores and** reasoning
 - ↓
- 4. **Pricing** Agent → quote with **breakdown**
 - ↓ (user confirms in app)
- 5. **Booking** Agent → reservation + conflict check
 - ↓ (**background**)
- 6. **Follow-up** Agent → **schedules** reminders
 - ↓ (on cancellation)
- 7. **Reschedule** Agent → re-runs ranking, auto-**books**
 - ↓ (after completion)
- 8. **Feedback** Agent → updates reputation
 - ↓ (if **dispute** filed)
- 9. **Dispute** Agent → resolution recommendation

9.2 Agent Implementation Pattern

Every agent follows this pattern:

```
type AgentInput = { ... };           // Zod schema
type AgentOutput = {
  data: { ... };                     // Zod schema
  reasoning: string;                 // Human-readable
  confidence: number;                // 0-1
};

async function agentFunction(input: AgentInput, traceCtx: TraceContext):
Promise<AgentOutput> {
  const startTime = Date.now();
  const prompt = buildPrompt(input);
  const response = await geminiClient.generate({
    model: "gemini-2.5-flash",
    contents: prompt,
    config: { responseSchema: outputSchema }
  });
  const parsed = parseAndValidate(response);
  traceCtx.recordStep({
    agent: "agent_name",
    input,
    output: parsed,
    reasoning: parsed.reasoning,
    model: "gemini-2.5-flash",
    durationMs: Date.now() - startTime,
```



```
    tokens: response.usage
  });
  return parsed;
}
```

9.3 Antigravity's Role

Antigravity is the development environment, not a runtime service. The team uses Antigravity to:

- Plan project structure (workplans for each day, exported as artifacts)
- Generate scaffolding code (RN screens, backend routes, agent prompt templates)
- Run autonomous tasks (e.g., "generate the mock provider dataset", "write the ranking score function")
- Produce artifacts (task lists, screenshots, browser recordings) that are submitted as proof

What gets submitted as Antigravity artifacts:

- The master workplan showing the 5-day breakdown
- Task plans showing how the team decomposed work
- Reasoning logs from Antigravity sessions where the AI helped architect the solution
- Screenshots/recordings of Antigravity managing parallel agent tasks during development
- Final implementation artifacts and verification screenshots

9.4 Agent Trace as a User-Visible Feature

Every booking has its full agent trace stored in the `agent_traces` table. The mobile app has a "How I decided" panel that renders this beautifully — not as raw logs, but as a step-by-step narrative:

1. I understood you need: AC repair, F-10, tomorrow 9am, budget Rs. 4000 (confidence: 89%)
2. I looked at 12 providers in or near F-10
3. I ranked the top 3 by distance, rating, specialization, and price
4. I picked Ali Khan AC Services because of [reasoning]
5. I priced it at Rs. 3200 with this breakdown: [breakdown]
6. I booked the 10am slot (no conflicts)

This panel directly addresses both **Agentic Reasoning (20%)** and **Antigravity Integration (25%)** because it makes the agentic logic visible.

10. Stress Test Scenarios

The brief requires at least one stress test demonstrated. We will demonstrate **two**, in the same demo video.

10.1 Hero: Provider Cancellation + Auto-Reschedule

Setup: User has just confirmed booking with Ali Khan AC Services for tomorrow 10am.

Trigger: Provider (visible via role-switch in same app) opens active job and taps "Cancel — Emergency."

System response:

- Customer notification: "Your provider had to cancel."
- Reschedule Agent runs in ~3 seconds (visible loading state).
- New provider offered: Bilal AC Repair (different but comparable).
- New confirmation appears in chat.

What it proves: Multi-step autonomous reasoning, action simulation, system state change, recovery from failure.

10.2 Secondary: Ambiguous Multilingual Input

Setup: User opens fresh chat, types: "Kal AC banwana hai jaldi"

System response:

- Intent Agent returns confidence 0.62 (below threshold)
- App shows clarification card: "I understood: AC service, urgent. Did you mean tomorrow (kal = tomorrow) or yesterday? And where should the technician come?"
- User taps "Tomorrow" and types "G-13"
- Flow resumes normally

What it proves: Multilingual robustness (15%), confidence-aware behavior, graceful handling of ambiguity.

10.3 Backup Stress Tests (if time)

If Day 5 finishes early, add **No Provider Available** scenario: user requests 3am slot, no providers in working hours, agent suggests next-best slot with reasoning. 30 seconds in demo. Adds resilience evidence.

11. Third-Party APIs & Services

Service	Purpose	Auth	Cost concern?
Google Gemini API	All LLM calls	API key	Free tier covers hackathon easily

Service	Purpose	Auth	Cost concern?
Google Maps — Places API	Sector geocoding (optional)	API key	Free tier
Google Maps — Distance Matrix API	Travel time between user and providers	API key	Free tier
Google Maps — Static Maps API	Visual route preview in en-route card	API key	Free tier
Google Cloud Run	Backend hosting (optional)	gcloud CLI	Free tier

12. Out of Scope (Explicit)

The following are **not** being built. Anyone proposing them must justify by referencing the rubric.

- Real payment processing (Stripe, JazzCash, EasyPaisa)
- Real WhatsApp Business API integration
- Real SMS sending (Twilio)
- Real push notifications (FCM) — in-app polling-based notifications used instead
- Provider onboarding flow with document verification
- Background check for providers
- Live chat between customer and provider
- Multiple service categories beyond AC technicians
- iOS-specific builds (Android only for demo)
- Web app (mobile-only as per challenge)
- User profile editing (preferences are baked into mock data)
- Multi-city support beyond Islamabad
- Service categorization beyond basic/intermediate/complex
- AI-generated marketing copy or content
- Admin/moderator role
- In-app calling between customer and provider
- Bidding or counter-offer flows
- Subscription / membership tiers
- Loyalty points system beyond simple discount flag
- Referral program
- Provider photo verification with AI
- Real-time GPS tracking
- Voice transcription beyond Gemini's built-in audio understanding (only if time permits)

